SAP CAP Backend Developer

Practical Mock Exam 02 — CAP Sales & Customer Order Fulfillment

Unofficial practice scenario • Not an actual or leaked SAP exam task • Suggested practice time: 180 minutes

Your assignment

A global enterprise requires an end-to-end CAP backend to manage customers, product catalog, sales order intake, pricing rules, inventory allocation, and order fulfillment. You have been assigned an existing or newly created CAP Node.js project. Implement the required data models, event handlers, custom operations, declarative security, and packaging descriptors, then verify full operational readiness.

Objective: Deliver a working, production-grade application that fulfills every requirement below. Writing code without verifying runtime behavior and transactional integrity is insufficient.

Task 1 — Data modeling (15 points)

Create the CDS data model for the following entities under namespace sales:

Entity	Required fields
Customers	ID, customerNumber, name, email, creditLimit, loyaltyTier, active
Products	ID, productCode, name, description, category, unitPrice, availableStock
SalesOrders	ID, orderNumber, orderDate, deliveryDate, status, priority, customer, shippingAddress
OrderItems	ID, itemNumber, quantity, unitPrice, discountPct, product, sales order

- Use appropriate CDS types and aspects (cuid, managed).
- Define custom enum types for OrderStatus (DRAFT, CONFIRMED, IN_DELIVERY, DELIVERED, CANCELLED), Priority (LOW, MEDIUM, HIGH, URGENT), and LoyaltyTier (SILVER, GOLD, PLATINUM).
- Model the relationships: Association to Customers and Products; Composition of many OrderItems with cascade delete.
- Model virtual non-persisted fields: itemTotal on OrderItems; netAmount, taxAmount, and totalAmount on SalesOrders.
- Apply constraints: mandatory annotations, default values, and range validations (e.g. quantity ≥ 1).

Task 2 — Expose the service (10 points)

- Create a service named SalesService (or SalesFulfillmentService) mounted at /odata/v4/sales.
- Expose Customers as a @readonly projection to preserve customer master data.
- Expose Products, SalesOrders, and OrderItems with full transactional capabilities.
- Enable deep navigation through the service: retrieve an order together with its items and nested product details.
- Ensure service document and EDMX metadata ($metadata) compile cleanly without errors or warnings.

Task 3 — Implement business logic (20 points)

- Reject an order item with a quantity less than one or null.
- Reject an order item if discountPct is negative or exceeds 30.00% (maximum allowable discount).
- Reject sales order creation if the customer is inactive or does not exist.
- Validate customer credit limit: reject order creation if customer open orders plus the new order exceed creditLimit.
- Check inventory availability: reject an order item if the requested quantity exceeds the product's availableStock.
- In the after('READ', 'SalesOrders') handler, calculate line totals (itemTotal = quantity * unitPrice * (1 - discountPct/100)), order netAmount, 10% taxAmount, and totalAmount dynamically.
- Implement CAP event handlers (before, on, after) and verify that invalid requests fail with HTTP 400.

Task 4 — Add custom operations (10 points)

- Implement a custom business operation named confirmSalesOrder (CDS action) accepting orderID.
- Verify that the order exists (return HTTP 404 if not found) and contains at least one order item.
- Enforce lifecycle state machine: only orders in DRAFT status can be confirmed; reject invalid transitions.
- Atomically deduct confirmed item quantities from product availableStock within the transaction.
- Update order status to CONFIRMED and return a meaningful result message.
- Implement a custom function named calculateCustomerDiscount accepting customerID and orderAmount.
- Query the customer loyalty tier (SILVER = 5%, GOLD = 10%, PLATINUM = 15%) and return discount rate and discounted payable total.

Task 5 — Authorization and access control (10 points)

- Define authorization roles: SalesRepresentative and SalesManager.
- Restrict execution of the confirmSalesOrder action strictly to authorized SalesManager users.
- Apply declarative annotations (@requires, @restrict): restrict order creation and reading to representatives and managers; restrict destructive updates.
- Configure development authentication in package.json with mock personas (representative, manager, unauthorized).
- Test both authorized and unauthorized requests to verify that HTTP 403 Forbidden is returned for forbidden actions.
- Prepare production XSUAA configuration in xs-security.json defining scopes, role templates, and role collections.

Task 6 — Persistence, CQL queries, and deep operations (10 points)

- Deploy the database schema to SQLite (db.sqlite) for local execution.
- Verify deep inserts: successfully create a SalesOrder with multiple nested OrderItems in a single transactional POST payload.
- Filter sales orders by customer ID and status ($filter=status eq 'CONFIRMED').
- Sort products by unit price descending and filter by category.
- Retrieve an order together with its related items and product details ($expand=Items($expand=product)).
- Ensure transactional atomicity: verify that invalid item data triggers a full rollback leaving no inconsistent order records.

Task 7 — Build and deployment (15 points)

- Prepare the application for SAP BTP Cloud Foundry deployment using Multi-Target Application (MTA) architecture.
- Configure mta.yaml with DB deployer module (hdb), Node.js service module (nodejs), and XSUAA security resource.
- Configure dual runtime environments in package.json (development SQLite vs production SAP HANA Cloud).
- Run the production build command (npx cds build --production) and verify artifacts in gen/srv and gen/db.
- Assemble the deployment archive (.mtar) using the Cloud MTA Build Tool (mbt build).
- Document the Cloud Foundry deployment workflow (cf deploy), service key bindings, and log diagnostics.
- Local success does not count as proof of successful cloud deployment.

Task 8 — Final verification and test suite (10 points)

- The CAP server starts cleanly without errors or warnings (cds watch).
- Service metadata ($metadata) and entity endpoints respond with valid OData v4 JSON.
- Valid data can be created, retrieved, updated, and deleted through deep OData operations.
- Invalid input (negative quantity, excessive discount, stock breach, credit limit breach) is rejected with HTTP 400.
- Custom action confirmSalesOrder updates status and decrements stock; custom function returns correct discount calculations.
- Authorization restrictions work: role-based gates block unauthorized users with HTTP 403.
- Build archive is generated, and all test scenarios are documented in an executable test collection.

Scoring rubric

Area	Points
CDS data modeling	15
Service exposure	10
Business logic	20
Custom operations (Action & Function)	10
Authorization & Access control	10
Persistence, queries & deep operations	10
Build and deployment (MTA & Cloud Foundry)	15
Verification & Test suite	10
TOTAL	**100**

This scoring rubric is created for practice; it is not SAP's official scoring scheme or passing threshold.

How to use this mock exam

Work in VS Code or SAP Business Application Studio. Implement Task 1 first, run the relevant checks, then continue task by task. Do not look for a ready-made solution before attempting the work. For official preparation, verify the current exam format and scope on SAP's certification page.

Official references: https://learning.sap.com/certifications/sap-certified-associate-backend-developer-sap-cloud-programming-model and https://learning.sap.com/helpcenter/certification-support/system-based-assessments-faq